

Fase 1: Configuração do Ambiente e Boot em Emulação

A etapa inicial foca em estabelecer o alicerce de software básico para a arquitetura alvo, garantindo que o sistema operacional possa inicializar mesmo antes da disponibilização do hardware físico.

- **Adoção do Buildroot:** A construção de uma distribuição Linux embarcada do zero é altamente complexa; portanto, utilizar-se-á o sistema de *build* automatizado Buildroot. Ele será responsável por compilar o kernel, o *bootloader* e o sistema de arquivos raiz (*RootFS*).
- **Ecossistema de Boot (OpenSBI e U-Boot):** Para processadores RISC-V, o processo de boot exige camadas de privilégio. Deve-se compilar e integrar o OpenSBI (*Supervisor Binary Interface*), que atua no modo *Machine*, e o U-Boot, que carregará o kernel no modo *Supervisor*.
- **Validação via QEMU:** O primeiro marco de sucesso desta fase é gerar uma imagem genérica do Linux para a arquitetura de 32 bits (RV32IMA). O sistema deverá ser inicializado com sucesso no emulador QEMU, validando a integridade do kernel, do *bootloader* e do terminal de acesso antes da integração com o FPGA físico.

Fase 2: Configuração do Kernel e Geração da Toolchain

O sistema operacional atuará como a camada intermediária entre a aplicação de inteligência artificial e o acelerador físico. Esta fase prepara o ambiente para os demais membros da equipe de software.

- **Exportação da Cross-Compiler Toolchain:** A aplicação de inferência, escrita na linguagem C, não pode ser compilada nativamente no host (arquitetura x86). O Buildroot deverá ser configurado para gerar e exportar a *Cross-Compiler Toolchain* (ex: `riscv64-unknown-linux-gnu-gcc`), garantindo a execução do binário na arquitetura RISC-V. Esta ferramenta será entregue ao responsável pela aplicação (*User Space*).

+1

- **Parametrização do Kernel (menuconfig):** O Kernel Linux precisará de customizações específicas para o projeto. É mandatório habilitar o suporte a Módulos de Kernel Carregáveis (*Loadable Kernel Modules* - LKM), pré-requisito para o desenvolvimento do driver de comunicação.
- **Suporte a Temporizadores de Alta Resolução:** Para garantir o referencial comparativo e a métrica central do projeto, o kernel deve suportar temporizadores precisos. Isso viabilizará o uso de funções da biblioteca `<sys/time.h>` (`gettimeofday()`) para cronometrar a operação com precisão de milissegundos ou microssegundos.

+1

Fase 3: Construção e Integração da Device Tree (DTS)

A *Device Tree* é a estrutura de dados que descreve os componentes físicos de hardware para o kernel Linux, permitindo que o sistema operacional saiba onde os periféricos estão localizados na memória.

- **Recepção do Mapa de Memória:** O trabalho nesta etapa depende do documento técnico fornecido pela equipe de hardware, que especificará o Endereço Base da NPU no hardware físico e os *offsets* de cada registrador.
- **Mapeamento do Acelerador:** O arquivo *Device Tree Source* (DTS) da placa deverá ser editado para incluir o nó ("*node*") correspondente à NPU Ternária.
- **Viabilização da Comunicação:** Este documento é o requisito fundamental para que o desenvolvedor do Driver do Kernel possa iniciar a comunicação com o periférico físico. O mapeamento correto garantirá que o driver acesse os registradores de controle e dados por meio de *Memory-Mapped I/O* (MMIO).

+1

Fase 4: Sistema de Arquivos (RootFS) e Deploy Físico

A fase final consolida a infraestrutura do software para a execução real do *benchmark* no *System-on-Chip* (SoC) sintetizado no FPGA.

- **Configuração de Armazenamento:** Para a leitura dos *datasets* de IA, o sistema de arquivos raiz (*RootFS*) deve ser expandido. É necessário habilitar suporte a sistemas de arquivos como FAT32 ou ext4 e configurar a montagem de dispositivos de armazenamento em massa (ex: cartões SD ou pendrives USB, a depender da interface da placa).
- **Integração SoC-Linux:** O *bitstream* gerado na fase de síntese e validação física atestará o funcionamento do hardware executando um firmware simples. Em seguida, a imagem completa do Linux (kernel, *bootloader*, *Device Tree* e *RootFS*) será gravada no meio de armazenamento físico e inicializada sobre o SoC RISC-V físico.
- **Testes de Integração:** O teste final desta frente de trabalho é garantir que, uma vez feito o boot físico, o ambiente Linux seja capaz de carregar o driver da NPU (arquivos *.ko*) e executar o binário do espaço de usuário sem falhas de paginação de memória.